In this lecture, we will build a `RAG pipeline using LangChain + Chroma`.

We will use a real-life example:

E-commerce Policy Assistant
A chatbot that answers customer questions using company policy documents such as return policy, refund policy, warranty policy, shipping policy, and cancellation policy.

This is a perfect RAG use case because the answer should not come from the LLM's general memory. It should come from the company's latest internal documents.

LangChain's official retrieval docs explain that LLMs have two major limitations: they have a finite context window and static training knowledge. Retrieval solves this by fetching relevant external knowledge at query time and using it as context for generation.

High-Level RAG Flow

A normal LLM flow looks like this:

```
User Question
    |
    v
LLM
    |
    v
Answer
```

`Problem: The LLM may not know your company-specific policy.`

A RAG flow looks like this:

```
User Question
    |
    v
Retriever searches relevant document chunks
    |
    v
Relevant chunks are added to prompt
    |
    v
LLM generates answer from retrieved context
    |
    v
Grounded Answer
```

LangChain describes the indexing stage as Load → Split → Store, and the runtime stage as Retrieve → Generate.

Important Concepts

Document Loader
A loader reads external data and converts it into LangChain Document objects.

A Document has two important fields:

```
Document(
    page_content="actual text content",
    metadata={"source": "return_policy.md"}
)
```

The `page_content` is the actual text. The metadata stores useful information such as file name, page number, section name, URL, author, timestamp, etc.

LangChain's retrieval docs define document loaders as components that ingest data from external sources and return standardized Document objects.

Text Splitter
Large documents are difficult to retrieve and may not fit into the model context window. So we split documents into smaller chunks.

Example:

```
Long Refund Policy Document
        |
        v
Chunk 1: refund eligibility
Chunk 2: refund timeline
Chunk 3: refund exceptions
Chunk 4: payment reversal process
```

LangChain recommends `RecursiveCharacterTextSplitter` as a good default for generic text because it tries to preserve natural text structure while controlling chunk size.

Important parameters:

```
chunk_size=800
chunk_overlap=120
```

`chunk_size` controls the maximum chunk length. `chunk_overlap` keeps some repeated content between adjacent chunks, which helps when important information is split across chunk boundaries.

Embeddings
Embeddings convert text into numerical vectors.

Example:

```
"Refund will be processed in 7 business days"
      |
      v
[0.12, -0.43, 0.89, ...]
```

Similar meanings produce vectors that are close to each other.

OpenAI's embedding docs describe embeddings as vectors that measure relatedness between text strings; smaller distances suggest higher relatedness.

Vector Database
A vector database stores embeddings and allows similarity search.

In this session, we will use `Chroma`.

Chroma can run locally and can persist data using persist_directory, which means the vector store can be reused across runs instead of rebuilding every time.

Retriever
A retriever takes a user query and returns relevant documents.

Example:

```
retriever.invoke("How many days do I have to return electronics?")
```

Output:

```
return_policy.md chunk containing electronics return window
```

LangChain defines a retriever as an interface that returns documents for an unstructured query.

LCEL RAG Chain

LCEL stands for `LangChain Expression Language.`

It lets us compose the RAG flow like a pipeline:

`retriever | format_docs | prompt | llm | parser`

In practical terms:

```
Question
  |
Retriever
  |
Relevant Context
  |
Prompt
  |
LLM
  |
Answer
```

Install Dependencies
Create a virtual environment:

```
python3 -m venv venv
source venv/bin/activate
```

Install packages:

`pip install -U langchain langchain-openai langchain-community langchain-chroma langchain-text-splitters python-dotenv`

```
OPENAI_API_KEY=your_api_key_here
OPENAI_MODEL=gpt-4o-mini
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

`create_sample_corpus.py`

```
from pathlib import Path

BASE_DIR = Path("data/policies")
BASE_DIR.mkdir(parents=True, exist_ok=True)

documents = {
    "return_policy.md": """
# Return Policy

Customers can return most products within 10 days of delivery.
```

Electronics such as headphones, keyboards, smart watches, and mobile accessories can be returned only within 7 days of delivery.

Products must be unused, undamaged, and returned with original packaging.

Items marked as final sale cannot be returned.

If the product is defective, the customer must upload images or videos as proof before the return request is approved.
""",

    "refund_policy.md": """
Refund Policy

Refunds are initiated after the returned product passes quality inspection.

For prepaid orders, the refund is usually processed within 5 to 7 business days.

For cash-on-delivery orders, customers must provide bank account details or UPI ID.

Shipping charges are non-refundable unless the product was damaged, defective, or incorrectly delivered.

If the customer cancels before shipping, the full amount is refunded.
""",

    "shipping_policy.md": """
Shipping Policy

Standard delivery usually takes 3 to 5 business days in metro cities.

Delivery to remote areas may take 7 to 10 business days.

Customers receive a tracking link once the order is shipped.

If delivery fails because the customer is unavailable, the courier partner will make up to 2 additional delivery attempts.

Heavy appliances may require scheduled delivery.
""",

    "warranty_policy.md": """
Warranty Policy

Warranty coverage depends on the product category and manufacturer.

```
Electronic accessories come with a 6-month limited warranty.

Home appliances usually come with a 1-year manufacturer warranty.

Warranty does not cover physical damage, water damage, misuse, or unauthorized repair.

Customers must provide the invoice to claim warranty support.
""",

    "cancellation_policy.md": """
# Cancellation Policy

Customers can cancel an order before it is shipped.

Once the order is shipped, cancellation is not allowed. The customer may place
a return request after delivery if the item is eligible.

For prepaid orders cancelled before shipping, the full refund is processed within
3 to 5 business days.

Orders containing personalized or made-to-order products cannot be cancelled
after production starts.
"""
}

for filename, content in documents.items():
    file_path = BASE_DIR / filename
    file_path.write_text(content.strip(), encoding="utf-8")

print(f"Sample corpus created inside: {BASE_DIR}")
```

Step 1 - Load Documents Using LangChain Loaders

ingest.py

```
from pathlib import Path
from langchain_community.document_loaders import DirectoryLoader, TextLoader

DATA_DIR = Path("data/policies")

loader = DirectoryLoader(
    path=str(DATA_DIR),
    glob="**/*.md",
    loader_cls=TextLoader,
    loader_kwargs={"encoding": "utf-8"},
)
```

```
docs = loader.load()

print(f"Loaded {len(docs)} documents")

for doc in docs:
    print("Source:", doc.metadata.get("source"))
    print("Preview:", doc.page_content[:100])
    print("-" * 80)
```

Run:

```
python ingest.py
```

Expected output:

```
Loaded 5 documents
Source: data/policies/return_policy.md
Preview: # Return Policy Customers can return most products within 10 days...
```

`DirectoryLoader` loads multiple files from a folder.

`TextLoader` reads each file as text.

Each loaded file becomes a LangChain Document.

This is the first step in the RAG indexing pipeline.

Step 2 - Split Documents into Chunks

`ingest.py`

```
from pathlib import Path
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

DATA_DIR = Path("data/policies")

loader = DirectoryLoader(
    path=str(DATA_DIR),
    glob="**/*.md",
    loader_cls=TextLoader,
    loader_kwargs={"encoding": "utf-8"},
)
```

```
docs = loader.load()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
    add_start_index=True,
)

chunks = text_splitter.split_documents(docs)

print(f"Original documents: {len(docs)}")
print(f"Generated chunks: {len(chunks)}")

for i, chunk in enumerate(chunks[:3], start=1):
    print(f"\nChunk {i}")
    print("Source:", chunk.metadata.get("source"))
    print("Start index:", chunk.metadata.get("start_index"))
    print(chunk.page_content[:300])
```

We are not storing full documents directly in the vector DB.

We store smaller chunks because:

- Smaller chunks improve retrieval precision.
- They fit better into the LLM context.
- They reduce irrelevant context.
- They allow source-level citation.

Step 3 - Embed and Persist Vectors in Chroma

`ingest.py`

```
import shutil
from pathlib import Path
from dotenv import load_dotenv

from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

DATA_DIR = Path("data/policies")
CHROMA_DIR = Path("chroma_db")
COLLECTION_NAME = "ecommerce_policy_docs"

# Clean old DB for deterministic classroom demo.
# In production, you would usually do incremental updates instead.
```

```
if CHROMA_DIR.exists():
    shutil.rmtree(CHROMA_DIR)

loader = DirectoryLoader(
    path=str(DATA_DIR),
    glob="**/*.md",
    loader_cls=TextLoader,
    loader_kwargs={"encoding": "utf-8"},
)

docs = loader.load()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
    add_start_index=True,
)

chunks = text_splitter.split_documents(docs)

embedding_model = os.getenv("OPENAI_EMBEDDING_MODEL",
"text-embedding-3-small")

embeddings = OpenAIEmbeddings(model=embedding_model)

vector_store = Chroma(
    collection_name=COLLECTION_NAME,
    embedding_function=embeddings,
    persist_directory=str(CHROMA_DIR),
)

ids = vector_store.add_documents(documents=chunks)

print("Indexing completed successfully.")
print(f"Original documents: {len(docs)}")
print(f"Chunks stored: {len(chunks)}")
print(f"Chroma directory: {CHROMA_DIR}")
print(f"First 3 document IDs: {ids[:3]}")
```

python ingest.py

Expected output:

```
Indexing completed successfully.
Original documents: 5
Chunks stored: 5
```

```
Chroma directory: chroma_db
First 3 document IDs: [...]
```

This code does four things:

```
Load documents
      |
Split into chunks
      |
Create embeddings
      |
Store embeddings in Chroma
```

The important line is:

`persist_directory=str(CHROMA_DIR)`

This tells Chroma to save vectors locally so we can reuse them later. Chroma's LangChain docs show this same persistence pattern using persist_directory.

Step 4 - Load Existing Chroma DB and Create Retriever

`rag_app.py`

```
import os
from pathlib import Path
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_chroma import Chroma

load_dotenv()

CHROMA_DIR = Path("chroma_db")
COLLECTION_NAME = "ecommerce_policy_docs"

embedding_model = os.getenv("OPENAI_EMBEDDING_MODEL",
"text-embedding-3-small")

embeddings = OpenAIEmbeddings(model=embedding_model)

vector_store = Chroma(
    collection_name=COLLECTION_NAME,
    embedding_function=embeddings,
    persist_directory=str(CHROMA_DIR),
```

```
)

retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3},
)

query = "What is the return window for electronics?"

docs = retriever.invoke(query)

print(f"Query: {query}")
print(f"Retrieved documents: {len(docs)}")

for i, doc in enumerate(docs, start=1):
    print(f"\nDocument {i}")
    print("Source:", doc.metadata.get("source"))
    print("Start index:", doc.metadata.get("start_index"))
    print(doc.page_content)
```

`python3 rag_app.py`

Expected output should include the return policy chunk containing:

`Electronics ... can be returned only within 7 days of delivery.`

This file does not load and split the original corpus again.

It directly loads the persisted `Chroma DB`.

That means retrieval is reproducible across runs.

Chroma's LangChain integration supports converting a vector store into a retriever using `as_retriever(...)`.

Step 5 - Build LCEL RAG Chain

`rag_app.py`

```
from pathlib import Path
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

CHROMA_DIR = Path("chroma_db")
COLLECTION_NAME = "ecommerce_policy_docs"

llm_model = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
embedding_model = os.getenv("OPENAI_EMBEDDING_MODEL",
"text-embedding-3-small")

llm = ChatOpenAI(
    model=llm_model,
    temperature=0,
)

embeddings = OpenAIEmbeddings(model=embedding_model)

vector_store = Chroma(
    collection_name=COLLECTION_NAME,
    embedding_function=embeddings,
    persist_directory=str(CHROMA_DIR),
)

retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3},
)


def format_docs(docs):
    """
    Convert retrieved Document objects into a single string
    that can be inserted into the prompt.
    """
    formatted_chunks = []

    for i, doc in enumerate(docs, start=1):
        source = doc.metadata.get("source", "unknown")
        start_index = doc.metadata.get("start_index", "unknown")

        formatted_chunks.append(
            f"Source {i}: {source}, start_index={start_index}\n"
            f"{doc.page_content}"
        )

    return "\n\n".join(formatted_chunks)
```

```
prompt = ChatPromptTemplate.from_template(
    """
You are a helpful customer-support assistant for an e-commerce company.

Use ONLY the retrieved context to answer the user's question.

Rules:
1. If the answer is present in the context, answer clearly.
2. If the answer is not present in the context, say: "I don't know based on the provided documents."
3. Do not use outside knowledge.
4. Mention the source file name wherever possible.
5. Keep the answer concise and practical.

<context>
{context}
</context>

Question:
{question}

Answer:
"""
)

rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough(),
    }
    | prompt
    | llm
    | StrOutputParser()
)

questions = [
    "What is the return window for electronics?",
    "How long does delivery take in metro cities?",
    "Are shipping charges refundable?",
    "Can I cancel an order after it is shipped?",
    "Do electronic accessories have warranty?",
]

for question in questions:
    print("\n" + "=" * 100)
    print("Question:", question)
```

```
    print("-" * 100)

    answer = rag_chain.invoke(question)

    print(answer)
```

LCEL Chain Breakdown

This is the most important part:

```
rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough(),
    }
    | prompt
    | llm
    | StrOutputParser()
)
```

Let's break it down.

Part 1

`"context": retriever | format_docs`

This means:

```
Take user question
      |
Send it to retriever
      |
Get relevant documents
      |
Format documents into text
```

Part 2

`"question": RunnablePassthrough()`

This means:

Pass the original user question as it is

Part 3

`| prompt`

This fills the prompt with:

```
context = retrieved policy chunks
question = original user question
```

Part 4

`| llm`

This sends the final prompt to the model.

Part 5

`| StrOutputParser()`

This converts the model response into a normal string.

Step 6 - Grounding Comparison: With RAG vs Without RAG

`compare_grounding.py`

```
import os
from pathlib import Path
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

load_dotenv()

CHROMA_DIR = Path("chroma_db")
COLLECTION_NAME = "ecommerce_policy_docs"

llm_model = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
embedding_model = os.getenv("OPENAI_EMBEDDING_MODEL",
"text-embedding-3-small")

llm = ChatOpenAI(
    model=llm_model,
```

```
    temperature=0,
)

embeddings = OpenAIEmbeddings(model=embedding_model)

vector_store = Chroma(
    collection_name=COLLECTION_NAME,
    embedding_function=embeddings,
    persist_directory=str(CHROMA_DIR),
)

retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3},
)


def format_docs(docs):
    formatted_chunks = []

    for i, doc in enumerate(docs, start=1):
        source = doc.metadata.get("source", "unknown")
        start_index = doc.metadata.get("start_index", "unknown")

        formatted_chunks.append(
            f"Source {i}: {source}, start_index={start_index}\n"
            f"{doc.page_content}"
        )

    return "\n\n".join(formatted_chunks)


# Chain 1: LLM without retrieval
no_rag_prompt = ChatPromptTemplate.from_template(
    """
Answer the following customer-support question.

Question:
{question}

Answer:
"""
)

no_rag_chain = no_rag_prompt | llm | StrOutputParser()
```

```
# Chain 2: LLM with retrieval
rag_prompt = ChatPromptTemplate.from_template(
    """
You are a customer-support assistant.

Use ONLY the retrieved context to answer.
If the context does not contain the answer, say:
"I don't know based on the provided documents."

<context>
{context}
</context>

Question:
{question}

Answer:
"""
)

rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough(),
    }
    | rag_prompt
    | llm
    | StrOutputParser()
)


comparison_questions = [
    "What is the return window for electronics?",
    "How many delivery attempts will be made if I am unavailable?",
    "Can personalized products be cancelled after production starts?",
    "What is the warranty period for electronic accessories?",
    "Can I return a product after 45 days?",
]


for question in comparison_questions:
    print("\n" + "=" * 120)
    print("QUESTION:", question)
    print("=" * 120)

    no_rag_answer = no_rag_chain.invoke({"question": question})
    rag_answer = rag_chain.invoke(question)
```

```
retrieved_docs = retriever.invoke(question)

print("\nWITHOUT RAG:")
print(no_rag_answer)

print("\nWITH RAG:")
print(rag_answer)

print("\nRETRIEVED SOURCES:")
for doc in retrieved_docs:
    print("-", doc.metadata.get("source"))
```

Query 1
What is the return window for electronics?

Without RAG, the model may say:

Usually electronics can be returned within 30 days.

This may sound confident but is not grounded in our company documents.

With RAG, the model should say:

Electronics can be returned only within 7 days of delivery.

This is grounded because the answer came from `return_policy.md.`

Query 2
How many delivery attempts will be made if I am unavailable?

With RAG, the answer should say:

The courier partner will make up to 2 additional delivery attempts.

Source: `shipping_policy.md`

Query 3
Can personalized products be cancelled after production starts?

With RAG, the answer should say:

No. Personalized or made-to-order products cannot be cancelled after production starts.

Source: `cancellation_policy.md`

Grounding Quality Evaluation Criteria

Criteria	Without RAG	With RAG
Uses company policy?	No	Yes
Can cite source?	No	Yes
Risk of hallucination	High	Lower
Handles unknown questions	May guess	Can abstain
Good for dynamic data?	No	Yes
Good for public general knowledge?	Yes	Not always needed
Best use case	Generic Q&A	Domain-specific Q&A